

CRDTs and Databases: A Hands-On Tutorial

Part 2 — CRDTs in database systems

Paulo Sérgio Almeida Nuno Faria José Pereira

INESC TEC and University of Minho

VLDB 2026



INESC TEC



Universidade do Minho
uminho.pt

Key challenges and requirements

A survey of existing approaches

Conflict-free Replicated Data Views (CRDVs)

Rich, application-defined merge semantics

Merge is not an implementation detail: it is *application logic*.

Shopping cart

add-wins

`add(item) || remove(item)`

Retain the item: discarding a purchase is more costly than requiring its removal.

Access control list remove-wins

`grant(user) || revoke(user)`

Deny access: privilege should be granted conservatively.

Merge policy is not a property of the data type:

- ▶ **Data-dependent:** add-wins in the cart... *unless the item is discontinued*.
- ▶ **Purpose-dependent:** remove-wins on the ACL... *unless the query is an audit*, which must report every privilege ever granted.

R1

Merge semantics must be *expressible by the application* and selectable *at runtime, per query*. It must be neither fixed by the data type nor determined at build time.

Effective query optimization

A SQL database derives its performance from *choosing* how to answer a query.

What the optimizer needs to see

- ▶ the merge as *relational operators*, not a function call
- ▶ predicates it can push down
- ▶ columns it can project
- ▶ paths it can index

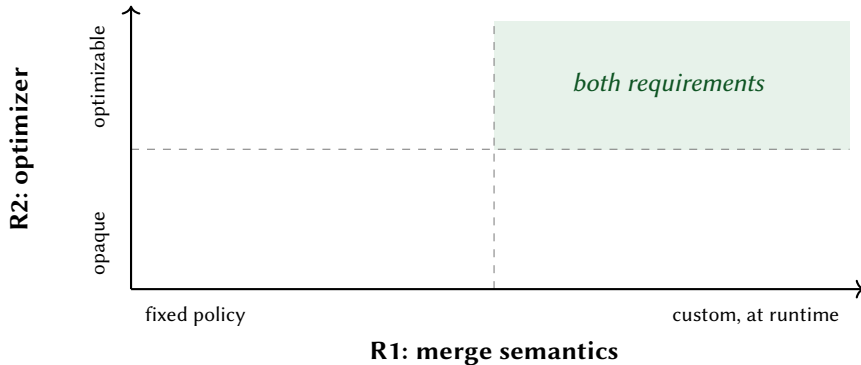
What this enables

- ▶ **configurable materialization:** eager for read-intensive, lazy for write-intensive
- ▶ operation reordering
- ▶ one plan spanning *application query* + *merge logic*

R2

Merge logic must fall within the scope of the query processor. Opaque values and mandatory eager materialization are both disqualifying.

The design space



Key challenges and requirements

A survey of existing approaches

Conflict-free Replicated Data Views (CRDVs)

A — Client-side CRDT libraries

R1 merge ~ R2 optimize ✗

The most mature CRDT implementations reside *outside* the database: comprehensive data types, but no query processor and no schema.

System	Data model	Sync	Conflicts	Implementation
Automerger	JSON document	operation-based (docs hold all changes ⇒ can ship state too)	LWW, multi-value	library, multi-lang (JS/Rust core, plus Swift, Python, C, Java)
Yjs	document (all types are <i>lists</i> inside)	op-based inserts, state- based deletes	highest client id (YATA)	library, multi-lang
Diamond Types	plain text only (sequence)	operation-based, (<code>clientID</code> , <code>seq</code>) per char	—	Rust + WASM (Node, browser)

Usage: collaborative editors and local-first applications, such as Zed, JupyterLab and MUTE. These libraries also serve as the backend of relational proposals (family D).

A — Client-side CRDT libraries

Automerge

```
let doc = Automerge.init()
doc = Automerge.change(doc, d => {
  d.cart = d.cart || []
  d.cart.push({sku: "A12", qty: 1})
})
// merge is a whole-document operation
const merged = Automerge.merge(mine, theirs)
```

Yjs

```
const doc = new Y.Doc()
const cart = doc.getArray('cart')
cart.push([{sku: "A12", qty: 1}])
// ship ops, or a state-vector diff
Y.applyUpdate(doc, update)
```

A document API, not a query language

- ▶ the unit of work is the **whole document**, loaded and mutated in memory
- ▶ updates are **method calls** on an object graph, not statements
- ▶ merge is an explicit operation on documents, invoked by the application
- ▶ reads are traversals, so there is no projection, selection or index

Verdict. R1 ~ fixed menu of types, no per-query policy. R2 ✗ no query engine is present.

B — Local-first databases with CRDT support

R1 merge ✗ R2 optimize ✗

An initial attempt at a database with native CRDT support, at the cost of the read path.

System	Data model	Sync	Conflicts	Mechanism
OrbitDB	key-value, docs, events	op-based, but each op carries the whole state	LWW	IPFS storage and libp2p pubsub. OpLog is an immutable Merkle-DAG (<i>Merkle-CRDTs</i>)
RxDB (CRDT plugin)	document	operation-based	highest creator id	ops accumulate in a dedicated schema field, with writes as MQL operators

The read path

OrbitDB answers a read by *iterating the log* until it finds the entry. **RxDB** replays the operation array stored with the document. Both materialize on *every* read, **with no alternative available**. This is the opposite of the configurable materialization required by R2.

Verdict. R1 ✗ a closed operator vocabulary. R2 ✗ materialization on read is mandatory.

Production-grade, server-side, with extensive type catalogues. The types are *predefined*, the values are *opaque*, and no query optimizer is involved.

System	Types	Sync	Notes
Riak KV	counters, sets, maps, flags, registers (<i>context-required</i>); HyperLogLog (<i>context-free</i>)	state-based	<i>context</i> = causal metadata the client carries. Without it a remove may fail or <i>over-remove</i>
Redis Active-Active	Counter → counter, Set → AW-OR-Set, Hash → map, String → LWW register	operation-based	standard Redis API , no dialect, strong eventual consistency
AntidoteDB	LWW-reg, MV-reg, counter, fat counter, integer, EW/DW-flag, G-Set, AW/OR-Set, RW-Set, G-Map, AW-Map, RR-Map	operation-based	the most extensive published set, with transactional CRDTs

The limit of built-in implementations

Robust implementations, but **no extensibility**. A predefined set of types, however extensive, remains *closed*: neither data-dependent nor purpose-dependent policies can be expressed.

C — Redis: transparent resolution behind the standard API

```
# ordinary Redis commands, no CRDB dialect
INCRBY cart:42:qty 1      # counter: sums
SADD   cart:42 "A12"      # AW-OR-Set
SREM   cart:42 "A12"      # loses to a
                        # concurrent SADD
SET    user:7:pw "... "  # LWW register
```

Write path: *prepare* (process locally, produce an *effect*)
→ distribute the effect to all instances → *apply*.
Exactly-once, FIFO, bi-directional mesh.

Concurrency test by vector clock

On receiving an update from *B* for object *X* at *A*:

- ▶ $x_{vc}[b] > x_{vc}[a]$ → new, apply
- ▶ $x_{vc}[b] < x_{vc}[a]$ → old, ignore
- ▶ otherwise → **concurrent**, resolve

Add-wins rules

SADD > SREM APPEND > DEL larger TTL wins

Fully transparent, but entirely predetermined: the semantics are those of the system, selected by the type of the key.

Verdict. R1 $\sim$ rich but closed. R2 $\times$ opaque values block fine-grained projection and selection.

D — Blob-in-a-row: a CRDT library inside a column

R1 merge ~ R2 optimize ✗

The most direct path from family A into SQL, which forgoes the advantages of the relational engine.

```
CREATE EXTENSION crdt;

CREATE TABLE docs (
  id    int PRIMARY KEY,
  doc   crdt.automerge    -- opaque blob
);

-- merge = library call on the blob
UPDATE docs
  SET doc = crdt.automerge_apply(doc, $1)
  WHERE id = 42;
```

pg_crdt (Supabase): experimental PostgreSQL extension with **Automerge (Rust) as the backend**

What the optimizer sees

doc
one opaque value

no projection
no selection
no index
no access-path choice

In addition, a constant serialization / deserialization cost on every access.

Verdict. R1 ~ depends on the CRDT library used R2 ✗ opaque values unreachable by the optimizer

The closest prior art to CRDVs, and the family in which R1 is lost.

System	Implementation	Conflicts	Interface
ElectricSQL	library (JS) + triggers & views, server-side on PostgreSQL	LWW	SQL
CRR (<i>Yu & Ignat, SMDs'20</i>)	library (Erlang)	LWW + counters (<i>never released</i>)	SQL
CR-SQLite	SQLite / libSQL loadable extension	per-column CRDTs	SQL
Dart <code>sql_crdt</code>	Dart library (→ <code>sqlite_crdt</code> , <code>postgres_crdt</code>)	HLC-ordered	SQL

A common approach in all four

Tag every row with a timestamp. Resolve with LWW. Materialize the relations eagerly.

Verdict. R1 ✗ map semantics R2 ~ relational model exposed to optimizer but merge outside the query processor

F — High-level declarative customization

R1 merge ~ R2 optimize X

The appropriate principle is that the application declares the semantics. The merge, however, remains outside the query processor.

AntidoteSQL provides an extended subset of SQL (“AQL”) over AntidoteDB:

```
CREATE UPDATE-WINS TABLE cart (  
  sku  VARCHAR PRIMARY KEY,  
  qty  COUNTER_INT,  
  note VARCHAR          -- LWW by default  
);  
-- rows:      AW / RW  
-- flags:     EW  
-- others:    LWW
```

Limitations: no *custom* rules, as the choice is restricted to AW/RW/EW/LWW. Optimization is limited to secondary indexes, and the non-standard interface makes use outside Erlang difficult.

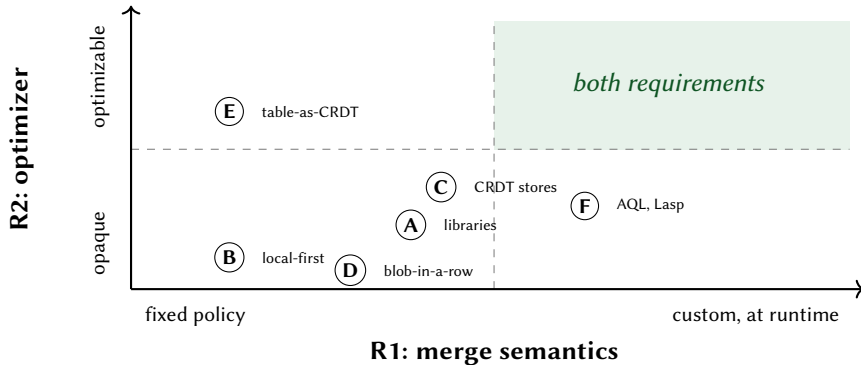
Lasp provides functional, application-level composition of CRDTs:

```
{ok, S1} = lasp:declare(set, awset),  
lasp:update(S1, {add, item}, self()),  
%% compose with functional operators  
lasp:map(S1, fun to_price/1, S2),  
lasp:fold(S2, fun sum/2, Total).
```

Custom resolution *is* expressible, but in Erlang and **at build time**, outside the database, so nothing is reusable across applications.

Verdict. R1 ~ declarative, but restricted to a closed rule set, or fixed at build time. R2 X no optimizing engine

The design space



Two research contributions identify this objective. Neither was realized in a production DBMS.

Data structures as queries: expressing CRDTs using Datalog

Kleppmann, 2018

Conjectures that a CRDT can be *specified as a query over a view containing all write operations*, and models it in Datalog:

```
% a register's current value = writes not overwritten by a later write
overwritten(A) :- write(A, _), write(B, _), causally_precedes(A, B).
current(V)      :- write(A, V), not overwritten(A).
```

Keep CALM and CRDT on Laddad, Power, Milano, Cheung, Crooks, Hellerstein. PVLDB 16(4), 2022

A vision for integrating CRDTs with the query planner.

Summary

	Family	The CRDT lives...	Examples	R1	R2
A	Client-side libraries	outside the DB entirely	Automerger, Yjs, D. Types	~	✗
B	Local-first databases	is the DB (a log)	OrbitDB, RxDB	✗	✗
C	CRDT-native datastores	in the engine, types fixed	Riak KV, Redis, AntidoteDB	~	✗
D	Blob-in-a-row	in a column, opaque	pg_crdt	~	✗
E	Table-as-CRDT	in the schema, policy fixed	ElectricSQL, CRR, CR-SQLite	✗	~
F	Declarative customization	in a high-level language	AntidoteSQL, Lasp	~	✗
G	Research conjectures	in a query / accessible to planner	CALM vision, Datalog CRDTs	✓	✓
→	CRDVs	in stacked SQL views	this tutorial	✓	✓

Key challenges and requirements

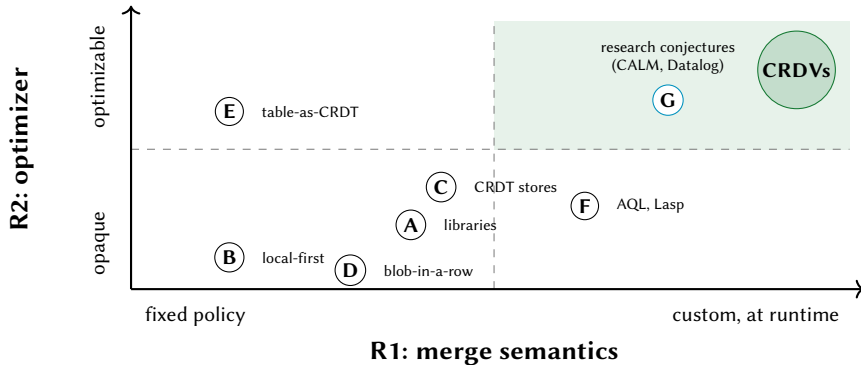
A survey of existing approaches

Conflict-free Replicated Data Views (CRDVs)

Claims

- (1) CRDVs are the only solution providing **custom conflict-resolution rules at runtime**, through **plain SQL**.
- (2) Those rules are **optimized together with the user's workload** in one plan covering application query *and* merge.
- (3) **Configurable materialization** over conflict-free replicated data, which addresses diverse read-intensive and write-intensive workloads.

CRDVs in the design space



The idea in one slide

Program convergent data in high-level SQL

- ▶ **merge semantics** are expressed as ordinary queries
- ▶ **materialization and indexing** are controlled to obtain the required performance
- ▶ the complexity of replication remains hidden
- ▶ existing **asynchronous replication** infrastructure performs the propagation

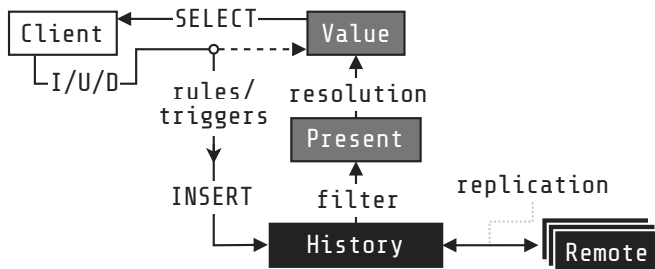
Three stacked layers, one per concern

- ▶ **History**: replication
- ▶ **Present**: causal filtering and GC
- ▶ **Value**: merge semantics

Why *views* and not tables

- ▶ no single materialization strategy is imposed
- ▶ the optimizer operates *across* layers, and across the application query

Architecture



Layer	Concern	Role
<i>Value</i>	semantics	current value, from user-defined resolution rules
<i>Present</i>	causality	exposes <i>all</i> concurrent versions
<i>History</i>	replication	partially ordered log, so concurrent writes coexist

Schema — what each layer stores

Layer	Columns
<i>Value</i>	key, data <i>the application's own schema, no metadata</i>
<i>Present</i>	key, data, op, site, lts, [pts]
<i>History</i>	key, data, op, site, lts, [pts]

- ▶ op: add or rmv (tombstone)
- ▶ site: originating site
- ▶ lts: logical timestamp (vector clock)
- ▶ pts: *optional* physical timestamp

Key uniqueness

In *Present* and *History*, key is **not unique**.
Several concurrent versions of the same key are stored simultaneously.

Writing

```
CREATE RULE insert_rule AS
ON INSERT TO Value DO INSTEAD
INSERT INTO History
SELECT key, data, 'add', siteId(),
      (t).lts, (t).pts
FROM nextTimestamp() AS t
```

UPDATE and DELETE follow the same strategy, also becoming INSERTS of new versions rather than in-place changes.

Every row inserted here is eventually replicated to *History* at every other site.

Deriving the vector clock

Query *Present* for the current per-site maxima, then increment the local entry:

$[7, 4, 1] \rightarrow [8, 4, 1]$ (at site 0)

Two consequences:

- ▶ correct timestamps **regardless of the materialization strategy**
- ▶ **no separate clock table**, which would be both overhead and a concurrency bottleneck

Filtering

```
CREATE VIEW Present AS
SELECT *
FROM History t1
WHERE NOT EXISTS (
  SELECT 1
  FROM History t2
  WHERE t1.key = t2.key
    AND t1.lts <> t2.lts
    AND vclock_lte(t1.lts, t2.lts)
)
```

Keep every version that was **not causally replaced**.

Straightforward SQL, not yet optimized.

This strategy, in which *Present* is computed purely by querying *History*, is called ***no-mat***.

When is a version superseded?

v_1 is replaced by v_2 when v_1 is comparable and less than v_2 . In other words, v_1 happened before v_2 .

Otherwise the two are **concurrent** ($v_1 \parallel v_2$) and **both are retained**.

Retaining concurrent versions is what allows the *Value* layer to apply *any* resolution rule.

Configurable materialization

Defining *Present* strictly as a view is wasteful, as only a small subset of *History* affects the result. It can instead be materialized, and the strategy is a configuration choice.

Strategy	<i>Present</i> is...	Maintained by	Trade-off
no-mat	a plain view over <i>History</i>	—	lowest write cost, but a full causal filter on <i>every</i> read
sync	a table	trigger on <i>History</i> , inside the writing transaction : DELETE the now-obsolete versions, INSERT the new one	lowest read cost, with the filter applied at write time
async	a view combining the materialized part with new <i>History</i> rows	a periodic procedure doing asynchronously what <i>sync</i> does synchronously	faster writes, as no filtering occurs on the write path, but reads must consider <i>Present</i> and <i>History</i> for <i>read-your-writes</i>

Takeaway

Identical semantics, three physical alternatives. The most suitable depends on the workload. The choice is made by the administrator and the optimizer rather than imposed by the data type.

Applications `SELECT` from *Value*, which exposes the application schema with no metadata. The remaining question is how concurrent versions are reconciled.

Common rules

- ▶ *AW / RW*: add- or remove-wins
- ▶ *MVR*: return all conflicting values (*defers to the application*)
- ▶ *LWW*: last physical timestamp wins
- ▶ *Avg / Max / Min*: rules over the *values themselves*
- ▶ *EW / DW*: for flags

Consequences of expressing them in SQL

- ▶ one **universal runtime implementation**: the same rules need not be reimplemented per application
- ▶ the **planner optimizes the rule**, together with the query that uses it
- ▶ the **output format** is also expressible: native rows, or JSON

Example — the *Present* layer

key	data	op	site	Its	pts
k1	1	add	1	[10, 1]	(122185,1)
k2		rmv	1	[13, 1]	(123447,1)
k3	3	add	1	[14, 1]	(123720,1)
k3		rmv	2	[9, 2]	(131233,1)
k4	4	add	1	[15, 1]	(124948,1)
k4	40	add	2	[9, 3]	(132198,1)

- ▶ **k1**: a single *add*
- ▶ **k2**: a *rmv* (tombstone)
- ▶ **k3**: concurrent *add* || *rmv*
- ▶ **k4**: concurrent *add* || *add*, values 4 and 40

Rows sharing a key appear together because their logical timestamps are concurrent: *Present* has already discarded all causally superseded versions.

Example — Last Writer Wins (LWW)

```
CREATE VIEW ValueLww AS
SELECT key, data
FROM (
  SELECT key, data, op, rank() OVER (
    PARTITION BY key
    ORDER BY pts DESC, site)
  FROM Present
) t
WHERE rank = 1 AND op = 'add'
```

A window function ranks the versions of each key by physical timestamp. Rank 1 is retained, and only if it is an add.

key	data
k1	1
k4	40

- ▶ **k3** is excluded, as its *rmv* is the more recent version
- ▶ **k4** resolves to **40**, the later write
- ▶ **site** acts as tie-breaker, so the result is deterministic at all sites

Example — Add Wins, Multi-Value Register, and merge value

AW + MVR: retain concurrent values

```
CREATE VIEW ValueAwMvr AS
SELECT key, array_agg(
  data ORDER BY data) data
FROM Present
WHERE op = 'add'
GROUP BY key
```

key	data
k1	[1]
k3	[3]
k4	[4, 40]

AW + Avg: merge value

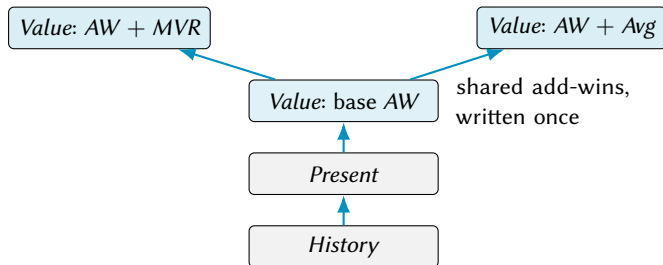
```
CREATE VIEW ValueAwAvg AS
SELECT key, avg(data) data
FROM Present
WHERE op = 'add'
GROUP BY key
```

key	data
k1	1
k3	3
k4	22

Both are *add-wins*: excluding rmv rows is what retains **k3** as 3.

They differ only in the treatment of **k4**: retaining both values, or averaging them to $\frac{4+40}{2} = 22$.

Composition



Reusability

Present and *History* are identical in all cases. Within *Value*, the base *add-wins* view is defined once, and *MVR* and *Avg* are built over it rather than repeating the logic.

Performance

The planner flattens and optimizes the **whole stack together**: base tables, causal filter, resolution rule, and the application's own query.

Transactions

CRDVs inherits the existing ACID concurrency control and recovery.

Principle: changes made in a transaction are visible to a remote site as a transaction.

Eventual consistency

Transactions from different sites may become visible **in different orders at different destinations**.

sync materialization

Operations at the same site are implicitly isolated by the local concurrency control.

async materialization

Two local transactions modifying the same item may both commit, as they only `INSERT` into *History* and therefore do not conflict.

To enforce local transactional semantics, an auxiliary table of the relevant keys can be used to induce the conflict (`SELECT ... FOR UPDATE`), or explicit locking can be applied.

Looking ahead — Transactional Causal Consistency

D. Pereira, F. Pereira, R. Lopes, N. Faria, P. Almeida, and J. Pereira. *Towards Abstractions for Composable Transactional Isolation*. Fourth International Workshop on Composable Data Management Systems (CDMS) @ VLDB, 2026. **Friday morning!**

Schema changes

The obstacle

Asynchronous logical replication in relational systems typically **does not replicate DDL**.

The available mechanism

DDL triggers are commonly available, so schema changes can be captured, processed and replicated in the same way as ordinary writes.

Why they cannot be applied unconditionally

Dropping a column at one site while another site is still writing to it results in an inconsistency.

The technique of F1 applies: one unsafe change is split into two safe ones, namely *deactivate* the column, then *remove* it. The second may execute only once every site has applied the first.

Portability

CRDVs requires four widely available features: **views** (for composition), **triggers/rules** (to capture and rewrite writes), **complex types** (to model metadata), and **asynchronous replication** (to ship *History*).

DBMS	Views	Triggers / DDL triggers	Complex types	Async. replication
PostgreSQL	Yes	Yes / Yes	Array, JSON	Yes
Oracle	Yes	Yes / Yes	Array, JSON	Yes
SQL Server	Yes	Yes / Yes	JSON	Yes
MySQL	Yes	Yes ^(a) / No	JSON	Yes

^(a) not supported on views.

Takeaway

No specialized functionality is required. CRDVs can be deployed on **mainstream SQL systems without modification**, requiring no new engine, fork or plugin.

Summary

R1 — merge semantics ✓

- ▶ rules are **ordinary SQL views**: *AW, RW, MVR, LWW, Avg, Max, EW...*
- ▶ defined by the **application**, not selected from a predefined set
- ▶ **several semantics simultaneously** over the same data, selected per query
- ▶ *Present* remains neutral (multi-value), so *Value* can implement any rule

R2 — optimization ✓

- ▶ merge logic is **relational operators**, fully visible to the planner
- ▶ **materialization is configurable**: *no-mat / sync / async*
- ▶ one plan spans **base tables + causal filter + rule + user query**
- ▶ indexes and access paths apply throughout, notably to vector timestamps

Reference

Solution	Data Model	Interface	Implementation	Sync.	Conflict Resolution	Optimization
Automerge	Document	Custom API	Library (Multi-lang)	Operation ^(a)	LWW, MVR	No
Yjs	Document	Custom API	Library (Multi-lang)	Both ^(b)	Highest Id	No
D. Types	Text	Custom API	Library (Rust, JS)	Operation	—	No
OrbitDB	Key-value	Custom API	Library (JS)	Operation ^(c)	LWW	No
RxDB	Document	MQL ^(d)	Library (JS)	Operation	Highest Id	Simple ^(e)
Riak KV	Key-value	Custom API	Internal (Erlang)	State	AW, EW, LWW, MVR	No
Redis	Key-value	Custom API	Internal (C)	Operation	AW, LWW	No
AntidoteDB	Key-value	Custom API	Internal (Erlang)	Operation	[A R]W, [E D]W, LWW, MVR	No
pg_crdt ^(f)	Relational	Custom API	PG Ext., Lib. (Multi-lang)	Operation	LWW, MVR	No
CRR	Relational	SQL	Library (Erlang)	State	LWW	Partial ^(g)
CR-SQLite	Relational	SQL	SQLite Extension	State	AW, LWW	Partial ^(g)
Dart sql_crdt	Relational	SQL	Library (Dart)	State	LWW	Partial ^(g)
ElectricSQL	Relational	SQL	Lib. (JS), Triggers+Views	State	LWW	Partial ^(g)
AntidoteSQL	Relational	AQL	Library (Erlang)	Operation	[A R]W, EW, LWW	Simple ^(e)
Lasp	Key-value	Custom API	Library (Erlang)	State	Custom (Erlang) ^(h)	No
CRDVs	Relational	SQL	Views+Triggers	State	Custom (SQL)	Yes

^(a) documents store all changes, so full state can also be replicated. ^(b) op-based inserts, state-based deletes. ^(c) op-based sync, but each operation carries the entire state. ^(d) MongoDB Query Language. ^(e) secondary indexes only, otherwise limited vs. major relational DBMSs. ^(f) Automerge as the backend. ^(g) conflict resolution is not considered by the optimizer. ^(h) rules must be defined at build time.

Links and references

Libraries (A)

- ▶ Automerger: automerger.org
- ▶ Yjs: github.com/yjs/yjs and [INTERNALS.md](#)
- ▶ Diamond Types: [josephg/diamond-types](https://josephg.github.io/diamond-types)

Local-first (B)

- ▶ OrbitDB: orbitdb.org
- ▶ RxDB CRDT plugin: rxdb.info/crdt.html

CRDT-native stores (C)

- ▶ Riak KV data types: docs.riak.com
- ▶ Redis Active-Active: redis.io/active-active
- ▶ AntidoteDB data types: antidotedb.gitbook.io

Relational blob-in-a-row (D)

- ▶ pg_crdt: supabase/pg_crdt

Relational table-as-CRDT (E)

- ▶ ElectricSQL: electric.ax (was electric-sql.com)
- ▶ CRR: Yu & Ignat, *Conflict-free Replicated Relations for Multi-Synchronous Database Management at Edge*, IEEE SMDS 2020
- ▶ CR-SQLite: [vln-io/cr-sqlite](https://vln-io.io/cr-sqlite)
- ▶ Dart sql_crdt: [cachapa/sql_crdt](https://cachapa.io/sql_crdt)

Declarative (F)

- ▶ AntidoteSQL: Lopes et al., *Antidote SQL: Relaxed When Possible, Strict When Necessary*, [arXiv:1902.03576](https://arxiv.org/abs/1902.03576), 2019
- ▶ Lasp: Meiklejohn & Van Roy, PPDP 2015

Research conjectures (G)

- ▶ Keep CALM and CRDT on: Laddad et al., [PVLDB 16\(4\)](#), 2022
- ▶ CRDTs using Datalog: Kleppmann, martin.kleppmann.com, 2018

CRDTs and Databases: A Hands-On Tutorial

Paulo Sérgio Almeida Nuno Faria José Pereira

INESC TEC and University of Minho

VLDB 2026



INESC TEC



Universidade do Minho
uminho.pt

This work is financed by National Funds through the Portuguese funding agency, FCT – Fundação para a Ciência e a Tecnologia; Paulo Almeida under the support UID/50014/2025 (DOI:10.54499/ UID/50014/2025); Nuno Faria and José Pereira by CMU–Portugal project ADAPQO (DOI:10.54499/2024.12585.CMU).